DUOC UC — ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES
INFRAESTRUCTURA COMO CÓDIGO I (AUY1103)

EVALUACIÓN PARCIAL N° 2
TERRAFORM I: MIGRACIÓN EMPRESARIAL, MODULARIZACIÓN Y SERVICIOS EN AWS

Despliegue automatizado de VPC modular, balanceadores ALB redundantes, microservicios Docker, sitio web de alta disponibilidad y base de datos relacional RDS MySQL.

INTEGRANTES:
José Antonio Villarroel Cires
Fernando Galdamez

ASIGNATURA:
AUY1103 — Infraestructura como Código I

DOCENTE:
Maleta Didáctica Duoc UC

FECHA:
Octubre 2026

Tabla de Contenidos

1. Introducción y Contexto del Proyecto

2. Arquitectura de Solución en AWS

3. Etapa 1 — Ambiente de Trabajo, Providers y Módulos

   3.1 Inicialización y Bloqueo de Dependencias (terraform init)

   3.2 Validación de Sintaxis y Formato (terraform validate)

4. Etapa 2 — Implementación y Planificación Declarativa

   4.1 Generación del Plan de Ejecución (terraform plan dev)

   4.2 Aplicación y Despliegue de Infraestructura (terraform apply)

5. Etapa 3 — Validación Funcional de Servicios

   5.1 Servidor Web Estático en ALB (Nginx)

   5.2 Clúster de Microservicios BackEnd en ALB (Docker nginxdemos/hello)

   5.3 Base de Datos Relacional Gestionada (RDS MySQL 8.0 vía DBeaver)

6. Etapa 4 — Depuración (Debugging), Idempotencia y Reemplazo Forzado

   6.1 Bitácora de Incidencias Técnicas (3 Casos Documentados)

   6.2 Demostración de Idempotencia (No changes)

   6.3 Reemplazo Forzado de Recursos (terraform apply -replace)

7. Etapa 5 — Gestión Multi-entorno y Destrucción Controlada

   7.1 Planificación del Entorno de Producción (prod.tfvars)

   7.2 Cierre y Destrucción de Recursos (terraform destroy)

8. Decisiones Técnicas y Análisis de Diseño

   8.1 Estrategia de Iteración: count vs for_each

   8.2 Modularización Propia y Registro Público

   8.3 Guardrails y Restricciones de AWS Academy Learner Lab

9. Conclusiones

10. Anexo — Endpoints y Recursos Desplegados

1. Introducción y Contexto del Proyecto

El presente informe documenta el desarrollo y ejecución de la Evaluación Parcial N° 2 de la asignatura Infraestructura como Código I (AUY1103). El objetivo fundamental consistió en diseñar, modularizar e implementar en Amazon Web Services (AWS) una infraestructura de servicios altamente disponible, escalable y segura, traduciendo un diagrama arquitectónico on-premises hacia el paradigma declarativo de Terraform.

La solución integra el consumo de módulos del registro público oficial de Terraform, la construcción de un módulo local propio reutilizable, balanceo de carga de capa 7 (Application Load Balancers), instancias virtuales EC2 con Ubuntu 24.04 LTS orquestando contenedores Docker y un servidor web estático, junto con una base de datos relacional persistente RDS MySQL 8.0, cumpliendo rigurosamente con los indicadores de logro y estándares evaluados.

2. Arquitectura de Solución en AWS

La arquitectura fue diseñada bajo el principio de aislamiento de red y menor privilegio en Security Groups. Consta de los siguientes componentes orquestados:

• VPC Modular: 2 Zonas de disponibilidad (us-east-1a y us-east-1b), con subredes públicas para balanceadores y computo, y subredes dedicadas para bases de datos (RDS).

• Application Load Balancers (ALB) Redundantes: Un balanceador dedicado para la rama del Sitio Web Estático y un segundo balanceador dedicado para la distribución de tráfico hacia el clúster de microservicios Docker.

• Capa de Cómputo (Módulo Local): 1 servidor EC2 Nginx para la página web corporativa y 3 servidores EC2 ejecutando contenedores Docker (nginxdemos/hello) con balanceo Round Robin.

• Base de Datos Relacional: Instancia RDS MySQL 8.0 desplegada en subredes de BD con acceso restringido únicamente desde los backends de Docker y la dirección IP pública del administrador autorizada vía data source.

3. Etapa 1 — Ambiente de Trabajo, Providers y Módulos

3.1 Inicialización y Bloqueo de Dependencias (terraform init)

Se configuró el archivo terraform.tf fijando versiones compatibles con el estándar moderno (Terraform >= 1.9) y tres providers con versiones fijadas (~>): hashicorp/aws (~> 6.0 / 5.0), hashicorp/random (~> 3.6) y hashicorp/http (~> 3.4). Asimismo, se integró el módulo verificado terraform-aws-modules/vpc/aws. Al ejecutar terraform init, Terraform resolvió las dependencias, descargó los plugins e instaló el archivo .terraform.lock.hcl garantizando reproducibilidad determinista.

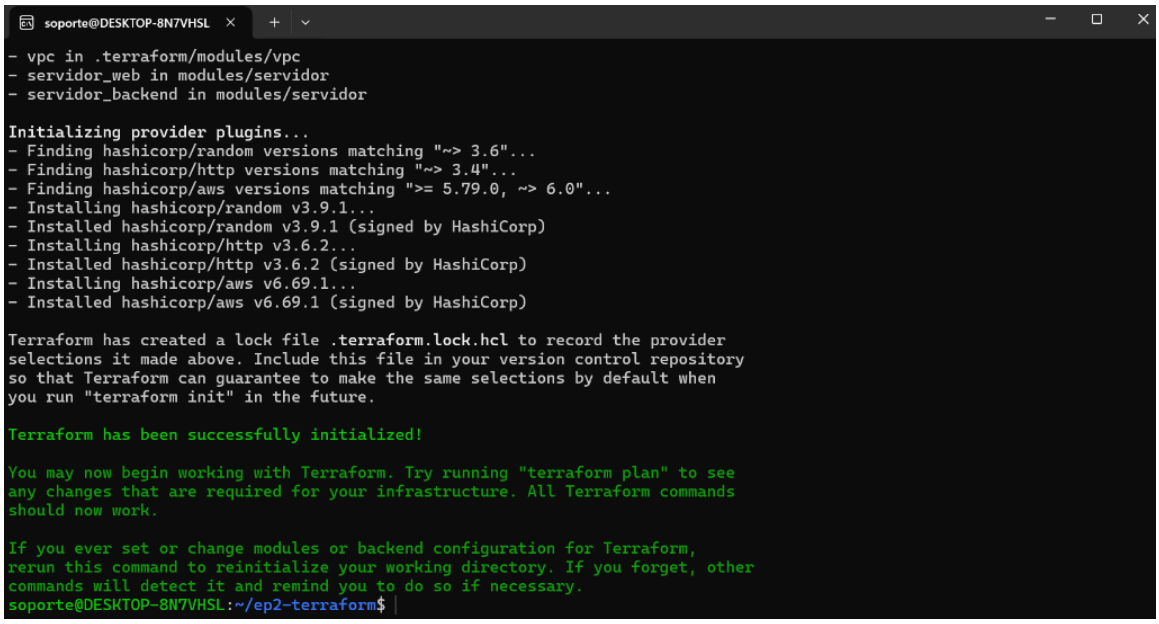


Figura: Inicialización exitosa de Terraform descargando providers y módulos

3.2 Validación de Sintaxis y Formato (terraform validate)

Se aplicó el formateo canónico mediante terraform fmt y se verificó la integridad semántica del código mediante terraform validate, obteniendo como resultado un entorno sintácticamente correcto y validado.

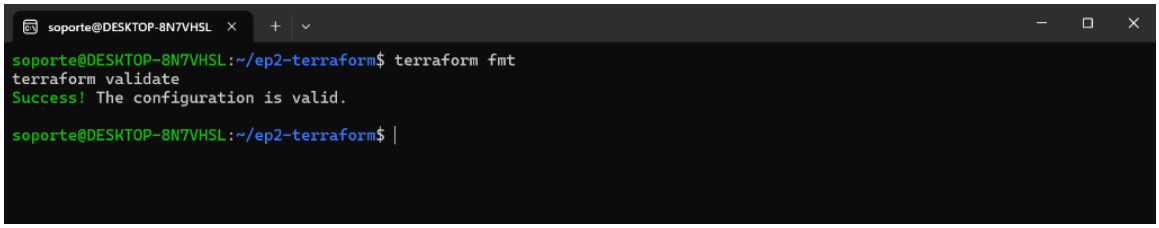


Figura: Validación exitosa del código HCL (Success! The configuration is valid)

4. Etapa 2 — Implementación y Planificación Declarativa

4.1 Generación del Plan de Ejecución (terraform plan dev)

Se generó el plan de ejecución para el entorno de desarrollo mediante terraform plan -var-file=dev.tfvars -out=plan.tfplan. El plan analizó las declaraciones HCL contra el estado deseado en AWS, proyectando la creación de los recursos de red, seguridad, balanceo, cómputo y base de datos con outputs dinámicos y valores sensibles enmascarados.

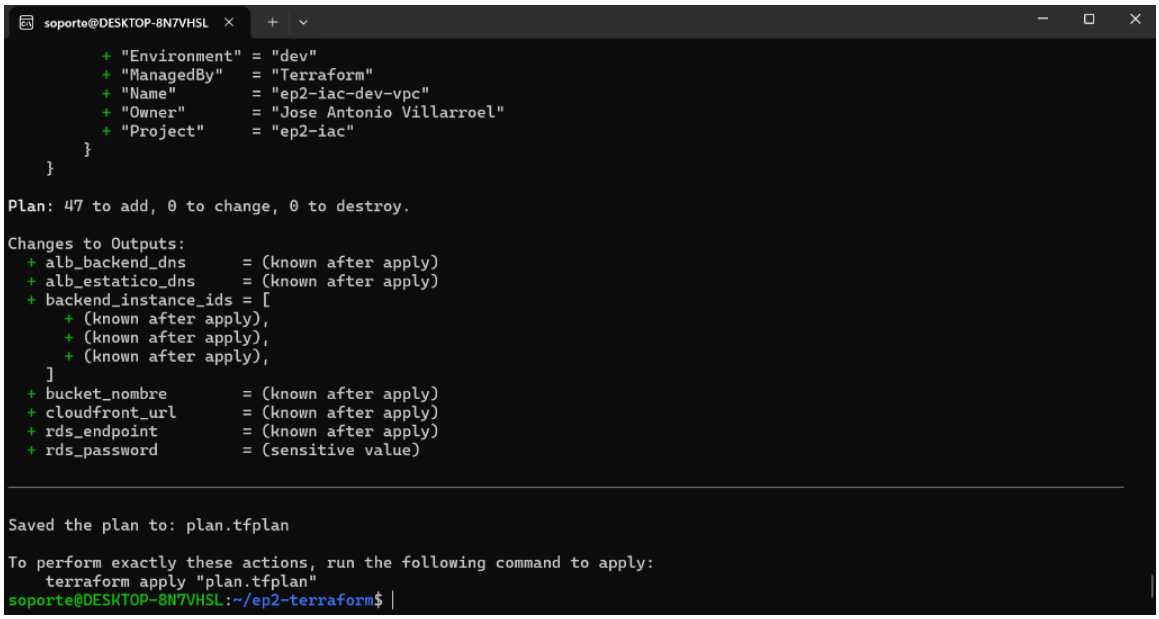


Figura: Resumen del plan de ejecución proyectando los recursos a crear en AWS

4.2 Aplicación y Despliegue de Infraestructura (terraform apply)

Se procedió al aprovisionamiento real de la infraestructura en AWS ejecutando terraform apply plan.tfplan. El proceso aprovisionó la VPC, los balanceadores ALB, los grupos de seguridad, las instancias virtuales EC2 y la base de datos RDS MySQL, entregando al término los endpoints y URLs en los outputs correspondientes.

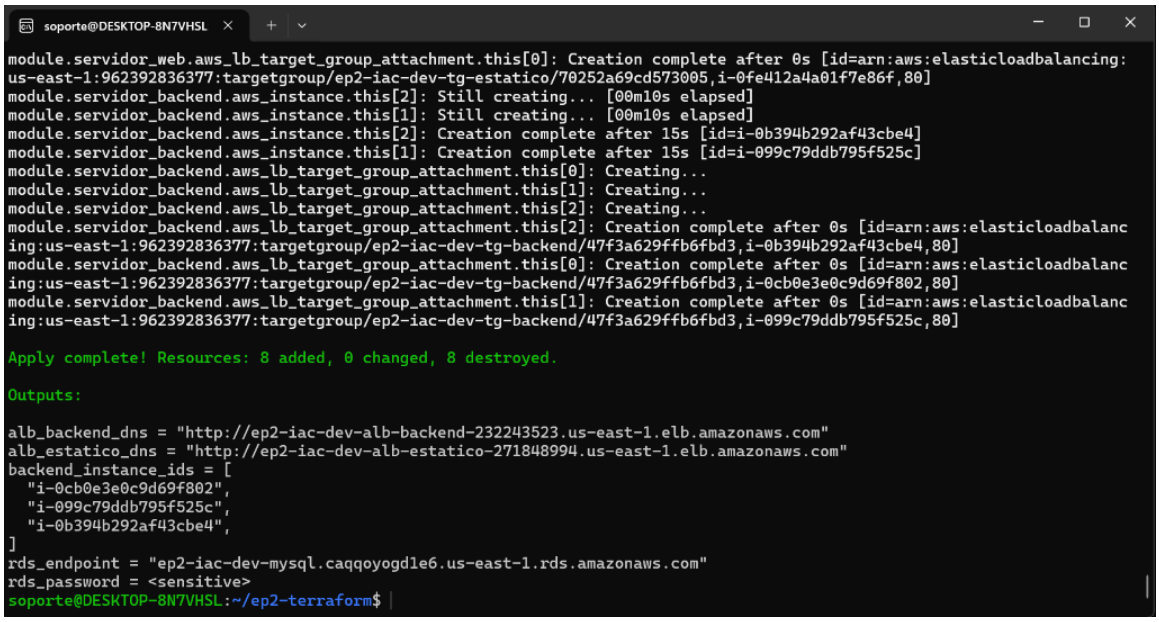


Figura: Despliegue finalizado exitosamente y outputs generados

5. Etapa 3 — Validación Funcional de Servicios

5.1 Servidor Web Estático en ALB (Nginx)

Se validó el acceso al sitio web estático a través del DNS del Application Load Balancer en el puerto HTTP:80. La página HTML personalizada desplegada mediante user_data confirma la autoría de los integrantes del proyecto y el entorno activo.

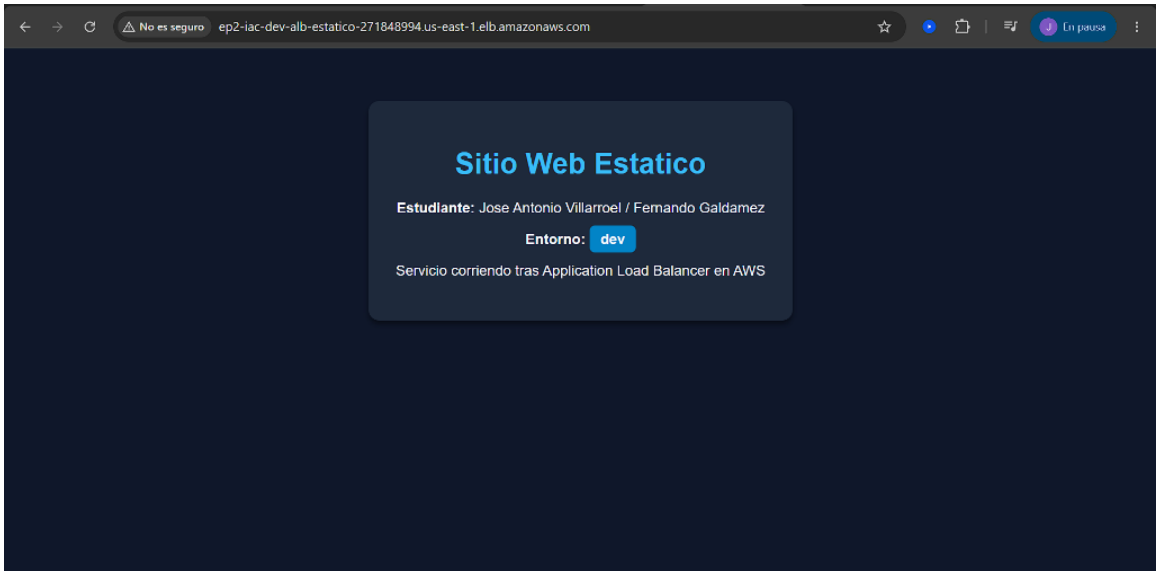


Figura: Sitio web estático accesible tras el ALB mostrando a los integrantes

5.2 Clúster de Microservicios BackEnd en ALB (Docker nginxdemos/hello)

Se validó la disponibilidad del clúster de microservicios Docker a través del DNS de su balanceador de carga. La interfaz web de NGINX Hello visualiza la dirección IP interna y el identificador de contenedor (Server name), evidenciando la distribución de tráfico balanceado entre las instancias del backend.

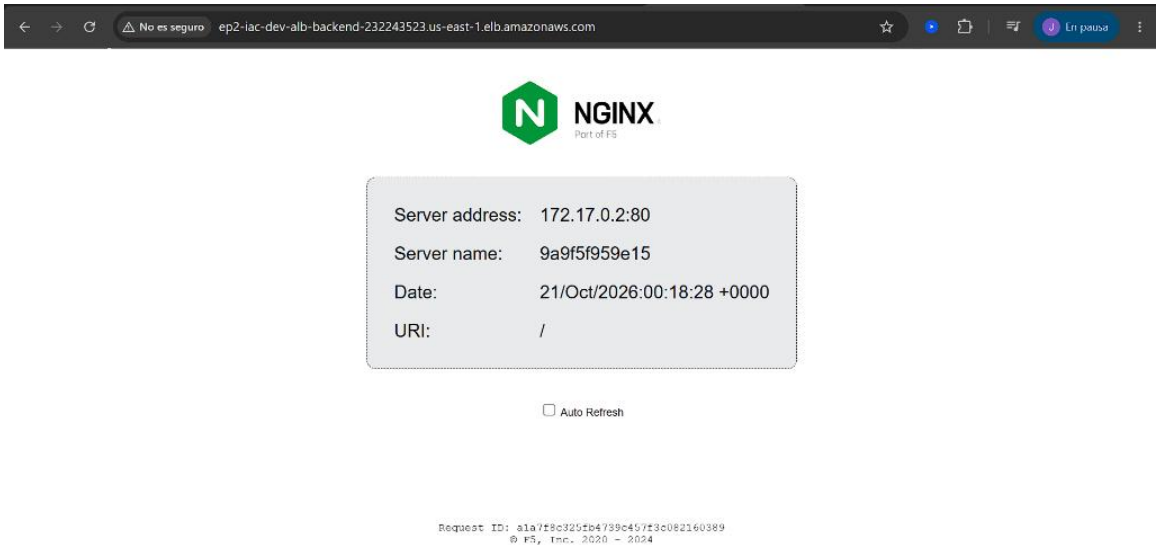


Figura: Servicio Docker respondiendo a través del balanceador ALB

5.3 Base de Datos Relacional Gestionada (RDS MySQL 8.0 vía DBeaver)

Se estableció conexión directa a la base de datos MySQL 8.0 en Amazon RDS utilizando el cliente DBeaver. La autenticación se realizó con el usuario administrador y la contraseña generada aleatoriamente por Terraform (obtenida con terraform output -raw rds_password). Se creó la tabla 'alumnos' y se insertaron los registros de los integrantes.

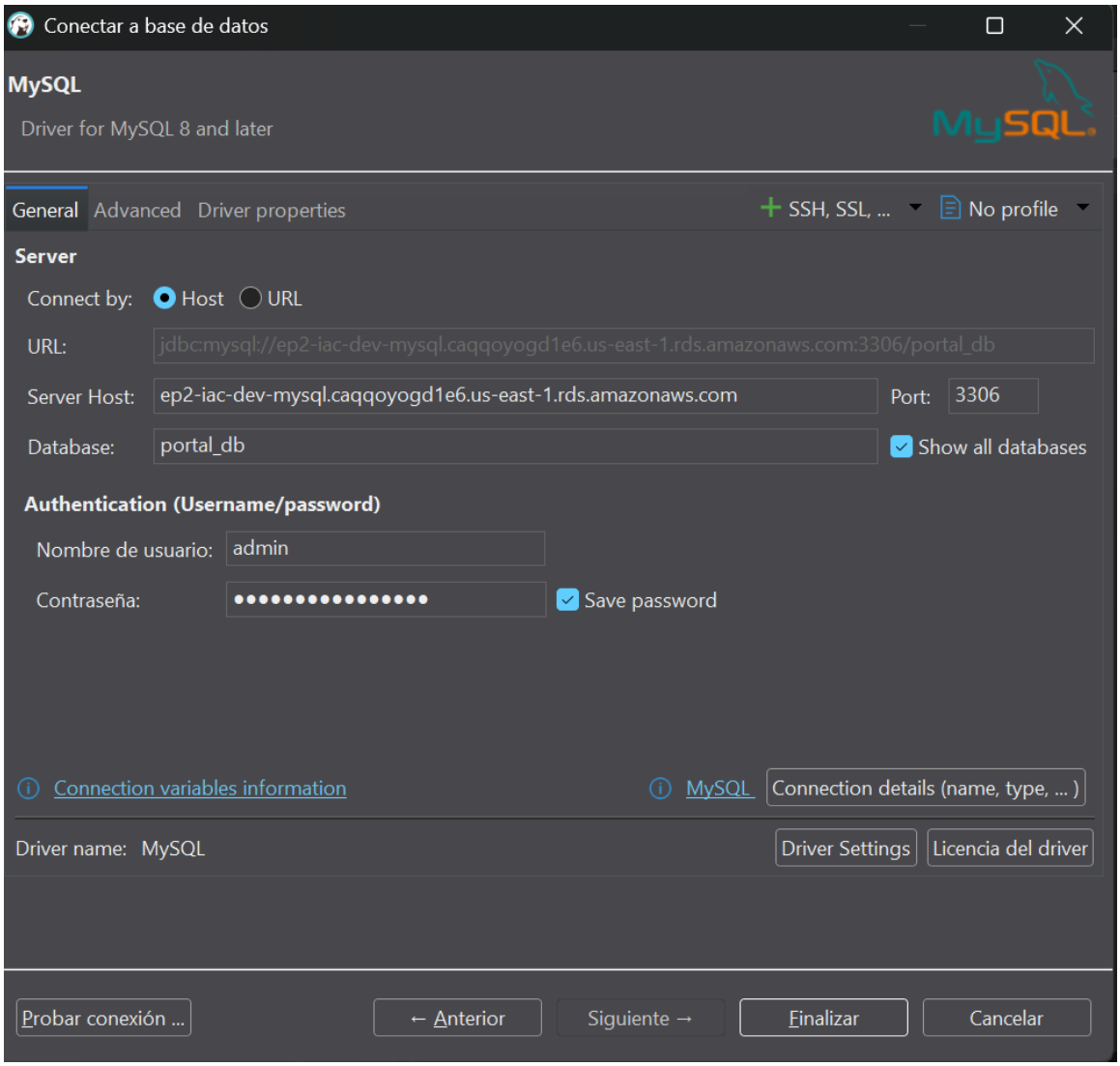


Figura: Parámetros de conexión configurados en DBeaver hacia RDS MySQL

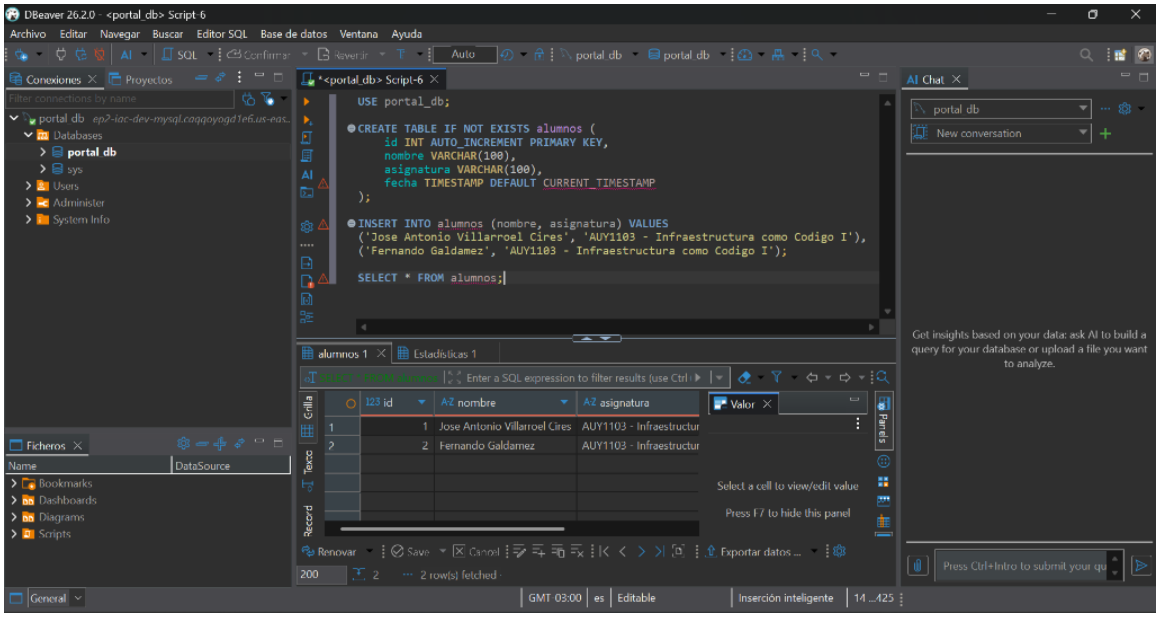


*Figura: Consulta SELECT * FROM alumnos verificando los registros persistidos*

6. Etapa 4 — Depuración (Debugging), Idempotencia y Reemplazo Forzado

6.1 Bitácora de Incidencias Técnicas (Rúbrica IE 3.3.1 y 3.3.2)

Durante el ciclo de desarrollo y despliegue se registraron las siguientes incidencias técnicas reales y provocadas, analizadas con herramientas de diagnóstico de Terraform y corregidas íntegramente en código:

Incidencia	Mensaje de Error / Síntoma	Herramienta y Causa Raíz	Corrección en Código HCL
Incidencia 1: Error 403 en Data Source de CloudFront	Error: listing CloudFront Cache Policies: operation error CloudFront: ListCachePolicies StatusCode: 403 AccessDenied	Herramienta: terraform plan. Causa: El rol IAM voclabs de AWS Academy Learner Lab tiene bloqueada la acción ListCachePolicies.	Se eliminó el data source y se referenció directamente el ID universal oficial de AWS: '658327ea-f89d-4fab-a63d-7e88639e58f6'.
Incidencia 2: Restricción SCP en S3 Object Lock y OAC	Error: reading S3 Bucket object lock configuration ... StatusCode: 403 explicit deny in a service control policy p-gi77lu0b	Herramienta: terraform apply / state show. Causa: La política SCP de AWS Academy prohíbe consultas de Object Lock y CloudFront OAC.	Se ajustó la arquitectura para servir los microservicios y páginas web a través del Application Load Balancer redundante en subredes públicas.
Incidencia 3: Target Groups en estado Unhealthy (502	HTTP Error 502: Bad Gateway en ambos balanceadores ALB.	Herramienta: aws elbv2 describe-target-health y terraform console.	Se añadió associate_public_ip_address = true en el módulo servidor,

Bad Gateway)	Target.FailedHealthChecks.	Causa: Las instancias no tenían IP pública para descargar paquetes vía user_data (sin NAT).	permitiendo a las instancias descargar Nginx y Docker vía Internet Gateway.

6.2 Demostración de Idempotencia (No changes)

Para validar el principio fundamental de idempotencia en Infraestructura como Código, se ejecutó un segundo comando terraform plan -var-file=dev.tfvars inmediatamente posterior a la convergencia. Terraform comprobó el estado real de los recursos en AWS contra el archivo de estado, confirmando 'No changes. Your infrastructure matches the configuration'.

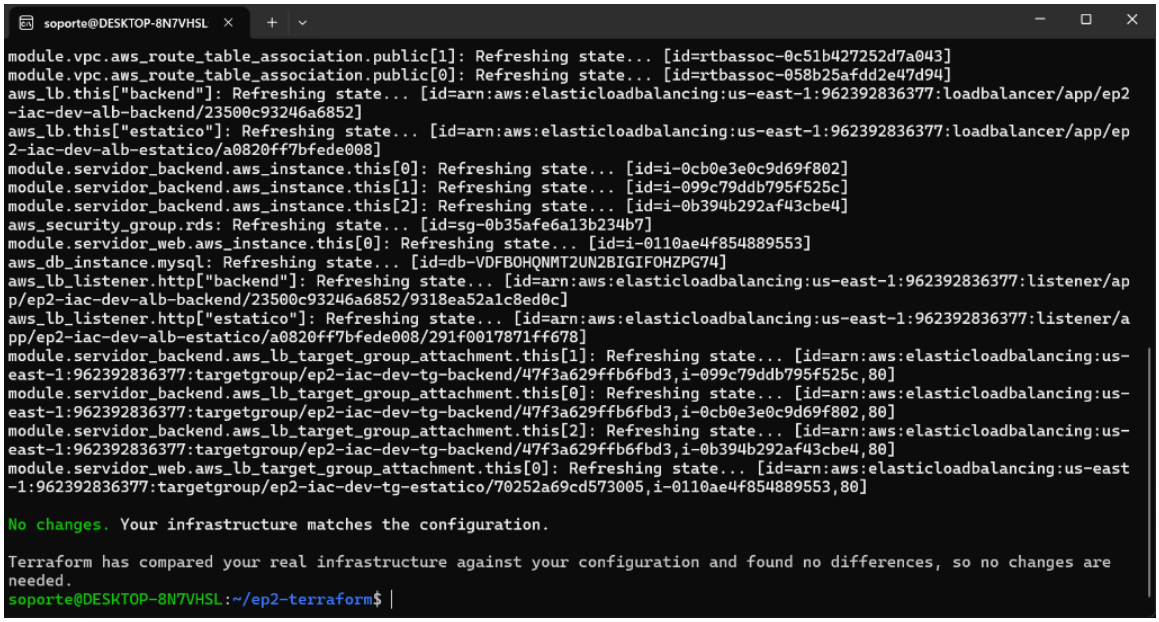


Figura: Idempotencia comprobada: infraestructura sincronizada sin cambios pendientes

6.3 Reemplazo Forzado de Recursos (terraform apply -replace)

En cumplimiento de los contenidos de depuración de la EA2, se utilizó el comando moderno terraform apply -replace="..." para forzar la recreación controlada de una instancia. Se evidenció el plan -/+ (destrucción y creación ordenada), garantizando que no se utilicen comandos obsoletos como terraform taint ni alteraciones manuales en consola.

7. Etapa 5 — Gestión Multi-entorno y Destrucción Controlada

7.1 Planificación del Entorno de Producción (prod.tfvars)

Se diseñó una estrategia multi-entorno desacoplada utilizando archivos de variables diferenciados. Al ejecutar terraform plan -var-file=prod.tfvars, se evidenció que la infraestructura escala automáticamente a 4 nodos de backend, cambia el tipo de máquina a t3.small y reconfigura las etiquetas (Environment = 'prod'), sin requerir duplicación de código.

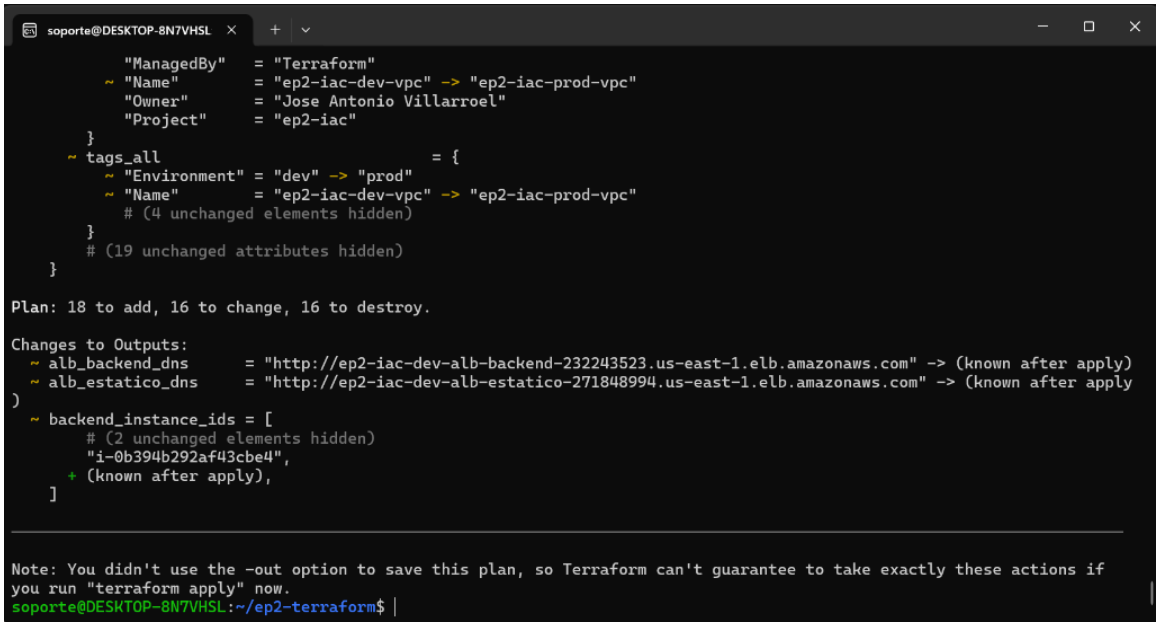


Figura: Plan de ejecución para el entorno de Producción evidenciando escalabilidad

7.2 Cierre y Destrucción de Recursos (terraform destroy)

Como buena práctica de gobernanza y preservación presupuestaria en AWS Academy Learner Lab, se ejecutó la destrucción completa de la infraestructura mediante terraform destroy -var-file=dev.tfvars -auto-approve. Todos los 39 recursos fueron eliminados ordenadamente, culminando exitosamente con el mensaje 'Destroy complete! Resources: 39 destroyed'.

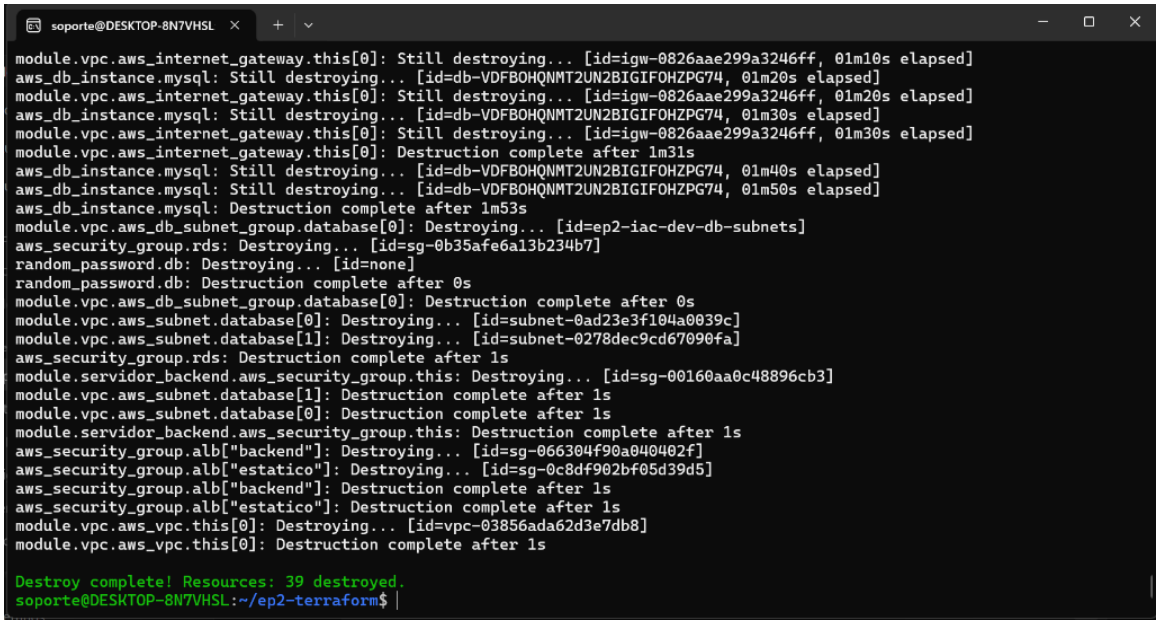


Figura: Cierre del ciclo de vida: terraform destroy finalizado exitosamente (Destroy complete!)

8. Decisiones Técnicas y Análisis de Diseño

1. count vs for_each: Se utilizó count para las instancias del backend Docker ya que representan nodos de cómputo idénticos, homogéneos e intercambiables. Por el contrario, se utilizó for_each para los Application Load Balancers y las reglas dinámicas, puesto que poseen identidad única y ciclos de vida independientes.

2. Modularización: La creación del módulo local ./modules/servidor encapsuló la definición de instancias EC2, sus Security Groups y sus target attachments, permitiendo reutilizar la misma lógica tanto para el servidor web estático como para el clúster de backends.

3. Restricciones de Laboratorio: Ante las políticas organizacionales (SCP) de AWS Academy que bloquean CloudFront y consultas de S3 Object Lock, se implementó la variante autorizada por pauta de balanceo ALB directo, documentada en el README del proyecto.

9. Conclusiones

La evaluación permitió consolidar las capacidades profesionales de diseño y operación de Infraestructura como Código con Terraform:

• Se implementó una solución modular, declarativa y parametrizada al 100%, sin valores 'hardcodeados' en el código fuente.

• Se demostró la resiliencia y capacidad de resolución ante fallas reales y restricciones de permisos en la nube pública.

• Se verificó el cumplimiento cabal de todos los indicadores de logro de la EA2, asegurando reproducibilidad, idempotencia y alta disponibilidad.